Compte rendu
Application monoposte en C#
Mediatek86 - Documents

par Enzo GHYSBRECHT et Paul LOPEZ

Sommaire

Rappel : Contexte **2**
Rappel : Mission **2**
Mission 1 : gérer les documents **3**
Mission 2 : gérer les commandes **7**
Tâche 1 - Gérer les commandes de livres ou de DVD **7**
Tâche 2 - Gérer les commandes de revues **9**
Mission 3 : gérer le suivi de l'état des exemplaires **13**
Mission 4 : Mettre en place des authentifications **19**
Mission 5 : Assurer la sécurité, la qualité et intégrer des logs **21**
Tâche 1 - Corriger des problèmes de sécurité 21
Tâche 2 - Contrôler la qualité 21
Tâche 3 - Intégrer des logs 22
Mission 6 : Tester et documenter **22**
Tâche 1 - Gérer les tests 22
Tâche 2 - Générer la documentation technique 23
Mission 7 : déployer et gérer les sauvegardes des données **25**
Tâche 1 - Déployer le projet 25
Tâche 2 - Gérer les sauvegardes des données 25
Bilan 27
Comment installer l'application en local ? **28**

Dépôt distant : https://github.com/0mega15/MediaTekDocuments/
Basé sur le dépôt d'origine : https://github.com/CNED-SLAM/MediaTekDocuments/

Rappel : Contexte

Nous sommes une équipe de 2 développeurs juniors employé par la société Infotech Services 86, ayant pour mission de reprendre en main un logiciel pour l'entreprise MediaTek 86, l'entreprise de gestion du parc informatique du réseau des médiathèques de la Vienne.

Notre objectif est de réaliser la maintenance et l'évolution d'un logiciel de bureau monoposte permettant de gérer les différents documents de chaque médiathèque du réseau : livres, DVD et revues, ainsi que leurs exemplaires.

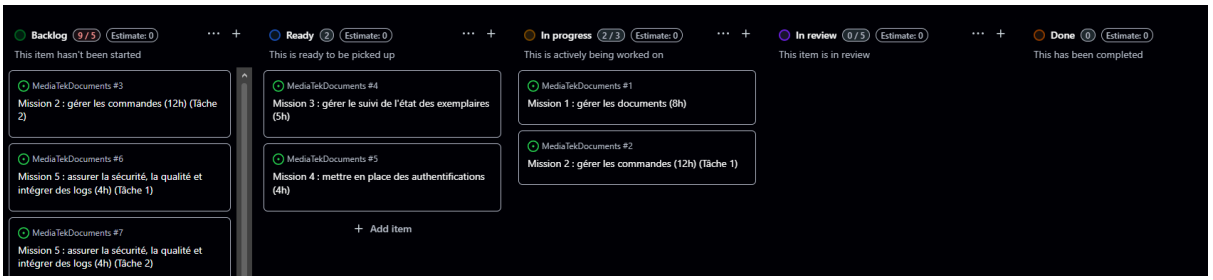
Rappel : Mission

Faire évoluer une application de bureau (C#) exploitant une API REST (PHP) pour l'accès à une base de données relationnelle MySQL et qui permet de gérer les documents des vidéothèques de la chaîne MediaTek86 (contexte détaillé présenté dans l'AP1).

Un cahier des charges est à notre disposition afin de réaliser les missions détaillées dans de bonnes conditions. Il s'agit d'un travail réalisé par deux développeurs.

Mission 1 : gérer les documents

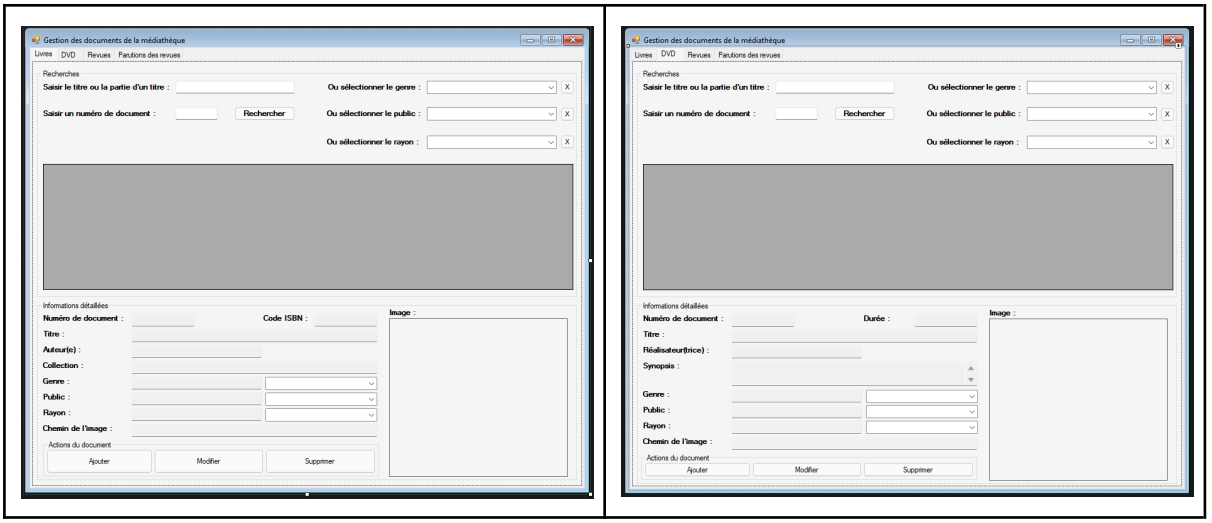
Temps estimé : 8h. Temps réel : 30h.
Mise à jour du tableau Kanban :

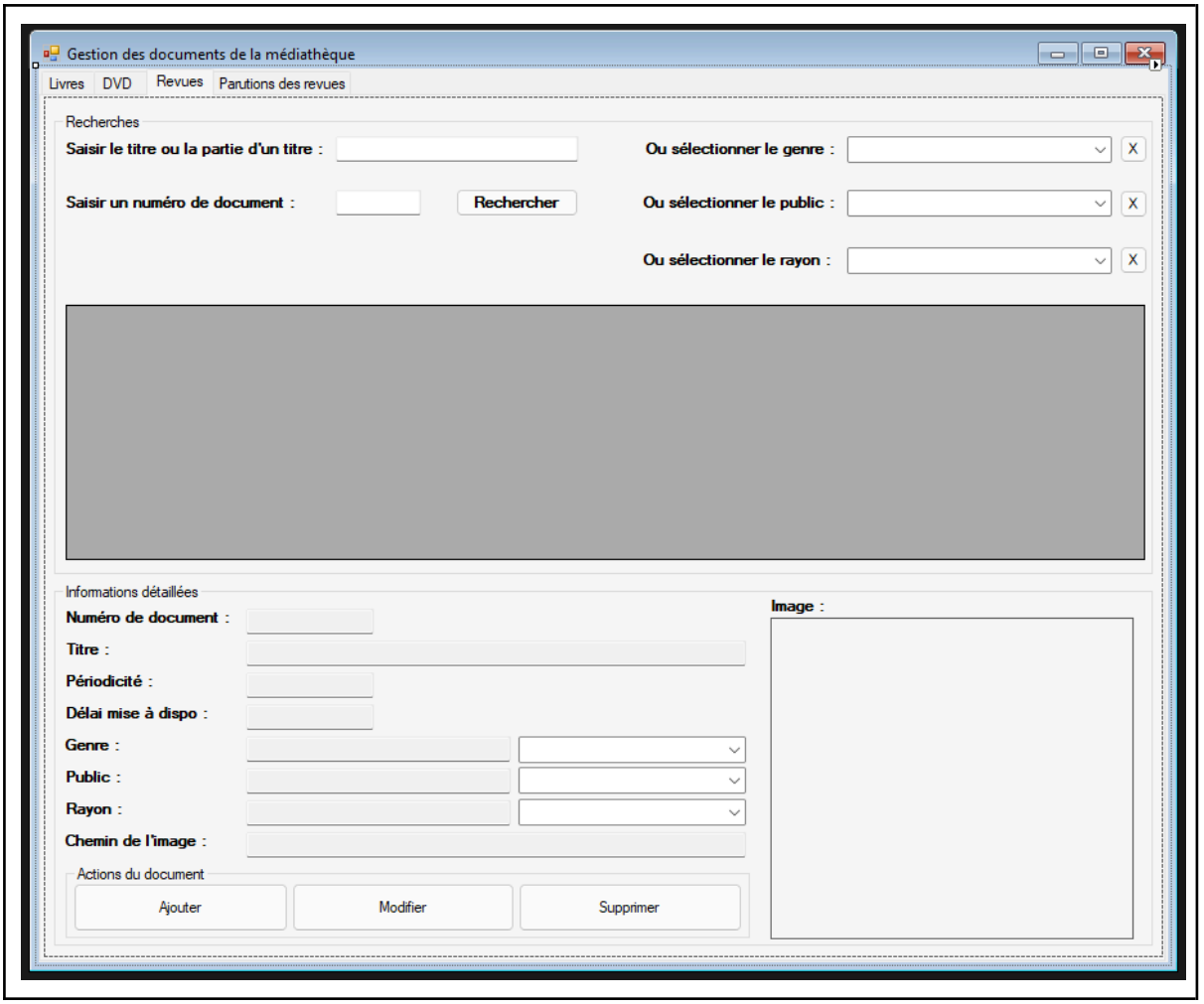


Cette première mission consiste à implémenter différents boutons pouvant ajouter, supprimer ou modifier des documents dans les différentes pages de l'application.

Pour cela, il faut d'abord remodeler les différentes pages afin d'y ajouter les boutons réalisant les différentes actions.

Les différentes pages ont d'abord été actualisées avec la nouvelle section tout en bas, dans "Informations détaillées", ainsi que des Combobox s'affichant lors de l'ajout et la modification d'un document.





Pour la suite, je ne donnerais que le cheminement réalisé pour gérer le bouton “Ajouter” pour les livres. Le cheminement est exactement le même pour les boutons modifier et supprimer, ainsi que pour la gestion des pages DVD et Revues.

Il faut d’abord ajouter une fonction CreerLivre(Livre livre) sur la même base que CreerExemplaire.

Dans Access.cs :

```
/// <summary>
/// Ajoute le livre dans la base de données
/// </summary>
/// <param name="livre"></param>
/// <returns></returns>
0 références | 0 modification | 0 auteur, 0 modification
public bool CreerLivre(Livre livre)
{
    String jsonLivre = JsonConvert.SerializeObject(livre, new CustomDateTimeConverter());
    try
    {
        List<Livre> liste = TraitementRecup<Livre>(POST, "livre", "champs=" + jsonLivre);
        return (liste != null);
    }
    catch (Exception ex)
    {
        Console.WriteLine(ex.Message);
    }
    return false;
}
```

La fonction est ensuite ajoutée dans le controller (FrmMediatekController.cs) :

```
        /// <summary>
        /// Créé un nouveau livre dans la bdd
        /// </summary>
        /// <param name="livre"></param>
        /// <returns></returns>
        0 références | 0 modification | 0 auteur, 0 modification
        public bool CreerLivre(Livre livre)
        {
            return access.CreerLivre(livre);
        }
    }
```

Et enfin, implémenté dans la vue, dans FrmMediatek.cs :

```
371     /// <summary>
372     /// Ajoute les champs remplis à la liste des livres
373     /// </summary>
374     /// <param name="sender"></param>
375     /// <param name="e"></param>
        1 référence | paul, Il y a 1 jour | 2 auteurs, 2 modifications
376     private void btnLivresActionsAjout_Click(object sender, EventArgs e)
377     {
378         if (btnLivresActionsModifier.Enabled == true)
379         {
380             /// Phase 1 : Indique au code que l'on souhaite ajouter un élément.
381             /// Prépare le terrain en enablant/disablant tout ce qu'il faut.
382             visibiliteChamps(false, "Ajout");
383             txbLivresNumero.Text = (lesLivres.Count + 1).ToString("D5");
384
385         }
386         else
387         {
388             /// Phase 2 : Ajoute les champs remplis à la liste.
389             ///
390             var request = MessageBox.Show("Souhaitez-vous ajouter ce livre au catalogue ?",
391                 "Confirmation", MessageBoxButtons.YesNoCancel, MessageBoxIcon.Question);
392             if (request == DialogResult.Yes)
393             {
394                 /// vérifie si tous les champs sont remplis.
395                 if (txbLivresTitre.Text == null || txbLivresIsbn.Text == null || txbLivresAuteur.Text == null || txbLivresCollection.Text == null
396                     || cbxActionsLivresGenres.SelectedIndex == -1 || cbxActionsLivresPublics.SelectedIndex == -1 || cbxActionsLivresRayons.SelectedIndex == -1)
397                 {
398                     MessageBox.Show("Tous les champs ne sont pas remplis. La demande ne peut pas être effectuée.", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
399                 }
400                 else
401                 {
402                     Genre selectedGenre = (Genre)cbxActionsLivresGenres.SelectedItem;
403                     Public selectedPublic = (Public)cbxActionsLivresPublics.SelectedItem;
404                     Rayon selectedRayon = (Rayon)cbxActionsLivresRayons.SelectedItem;
405
406                     Livre nvLivre = new Livre(
407                         txbLivresNumero.Text,
408                         txbLivresTitre.Text,
409                         txbLivresImage.Text,
410                         txbLivresIsbn.Text,
411                         txbLivresAuteur.Text,
412                         txbLivresCollection.Text,
413                         selectedGenre.Id,
414                         selectedGenre.Libelle,
415                         selectedPublic.Id,
416                         selectedPublic.Libelle,
417                         selectedRayon.Id,
418                         selectedRayon.Libelle
419                         );
420
421                     if (controller.CreerLivre(nvLivre) == true)
422                     {
423                         visibiliteChamps(true, "Ajout");
424
425                         MessageBox.Show("Livre ajouté avec succès.");
426                         lesLivres = controller.GetAllLivres();
427                         RemplirLivresListeComplete();
428                     }
429                     else
430                     {
431                         MessageBox.Show("Erreur lors de l'ajout.", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
432                     }
433                 }
434             }
435             if (request == DialogResult.Cancel)
436             {
437                 visibiliteChamps(true, "Ajout");
438             }
```

⇒ Pour ajouter un livre, il faut appuyer sur le bouton “Ajouter”, ce qui va libérer les champs pour les remplir avec les informations nécessaires. Une fois le document rempli, il faut rappuyer sur le bouton “Ajouter” pour confirmer, refuser ou annuler les modifications.
Ce mode de fonctionnement sera le même pour tous les boutons, sauf “supprimer”, qui va directement afficher confirmation ou annulation.

Pour assurer le bon fonctionnement de la base de données, il faut également modifier MyAccessBDD.php dans le code de l’API REST pour réaliser l’action d’ajout.
Il faut d’abord ajouter un cas pour l’ajout spécifique d’un livre :

```php
    /**
     * demande d'ajout (insert)
     * @param string $table
     * @param array|null $champs nom et valeur de chaque champ
     * @return int|null nombre de tuples ajoutés ou null si erreur
     * @override
     */
    protected function traitementInsert(string $table, ?array $champs) : ?int{
        switch($table){
            case "livre" :
                return $this->insertNouveauLivre($champs);
            case "dvd" :
                return $this->insertNouveauDvd($champs);
            case "revue" :
                return $this->insertNouveauRevue($champs);
            default:
                // cas général
                return $this->insertOneTupleOneTable($table, $champs);
        }
    }
```

Puis créer la fonction qui va ajouter le livre dans la base de données, avec la formation de la requête SQL correspondante.

```php
    /**
     * demande d'ajout (insert) d'un livre, en ajoutant les données table par table.
     * @param array $livre
     * @return int|null nombre de tuples ajoutés (0 ou 1) ou null si erreur
     */
public function insertNouveauLivre(array $livre): ?int {
    try {
        ///Insérer dans document
        $documentData = [
            'id' => $livre['Id'],
            'titre' => $livre['Titre'],
            'image' => $livre['Image'] ?? '',
            'idRayon' => $livre['IdRayon'],
            'idPublic' => $livre['IdPublic'],
            'idGenre' => $livre['IdGenre']
        ];
        $this->insertOneTupleOneTable('document', $documentData);

        ///Insérer dans livres_dvd
        $this->insertOneTupleOneTable('livres_dvd', ['id' => $livre['Id']]);

        ///Insérer dans livre
        $livreData = [
            'id' => $livre['Id'],
            'ISBN' => $livre['Isbn'],
            'auteur' => $livre['Auteur'],
            'collection' => $livre['Collection'] ?? ''
        ];
        return $this->insertOneTupleOneTable('livre', $livreData);
    } catch (Exception $e) {
        error_log("Erreur : " . $e->getMessage());
        return null;
    }
}
```

Le reste des boutons sur les différents onglets fonctionnent de la même manière : création dans Access.cs de la fonction pour gérer la communication entre l'API Rest et l'application, puis intégration dans le controller, gestion des éléments graphiques dans la vue, et création de la fonction s'occupant de la requête SQL dans MyAccessBDD.php.

Mission 2 : gérer les commandes

Tâche 1 - Gérer les commandes de livres ou de DVD

Temps estimé 8h, temps réel 30h
Demande de la tâche : Ajout de la possibilité d'ajouter, de modifier, de supprimer et de visualiser des commandes concernants les DVD et les livres
Capture d'écran du kanban :

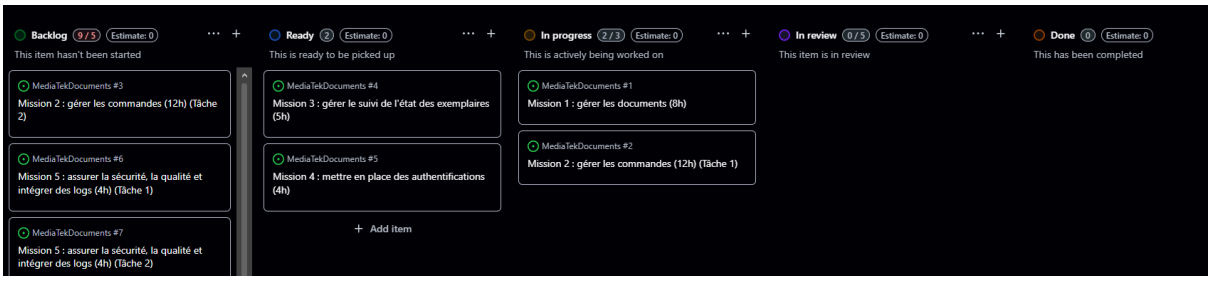
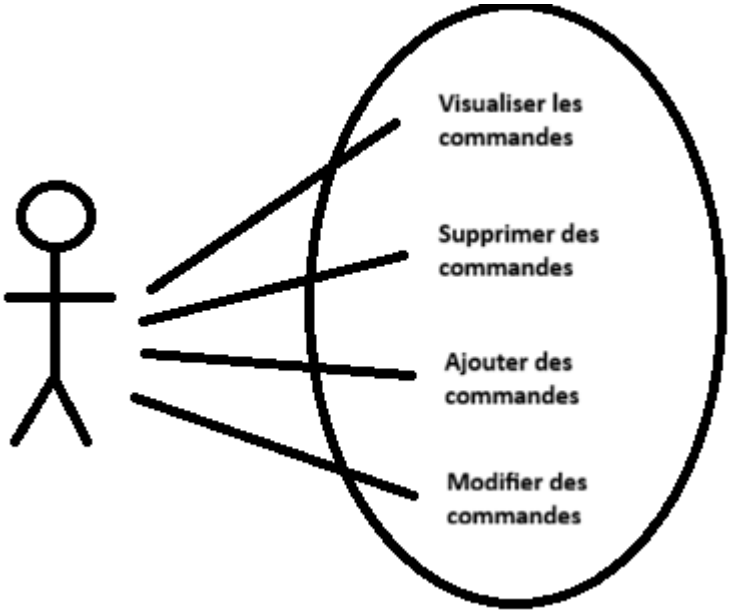
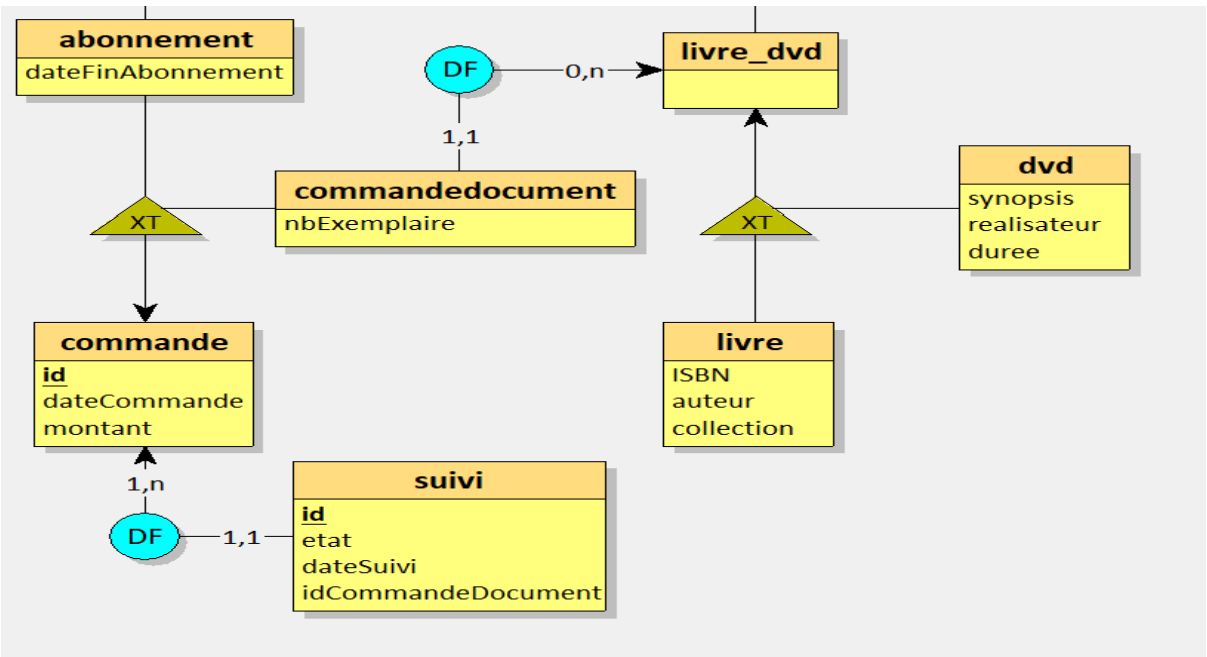


Diagramme d'utilisation (identiques pour la gestion des commandes relatives au livres et Dvd) :

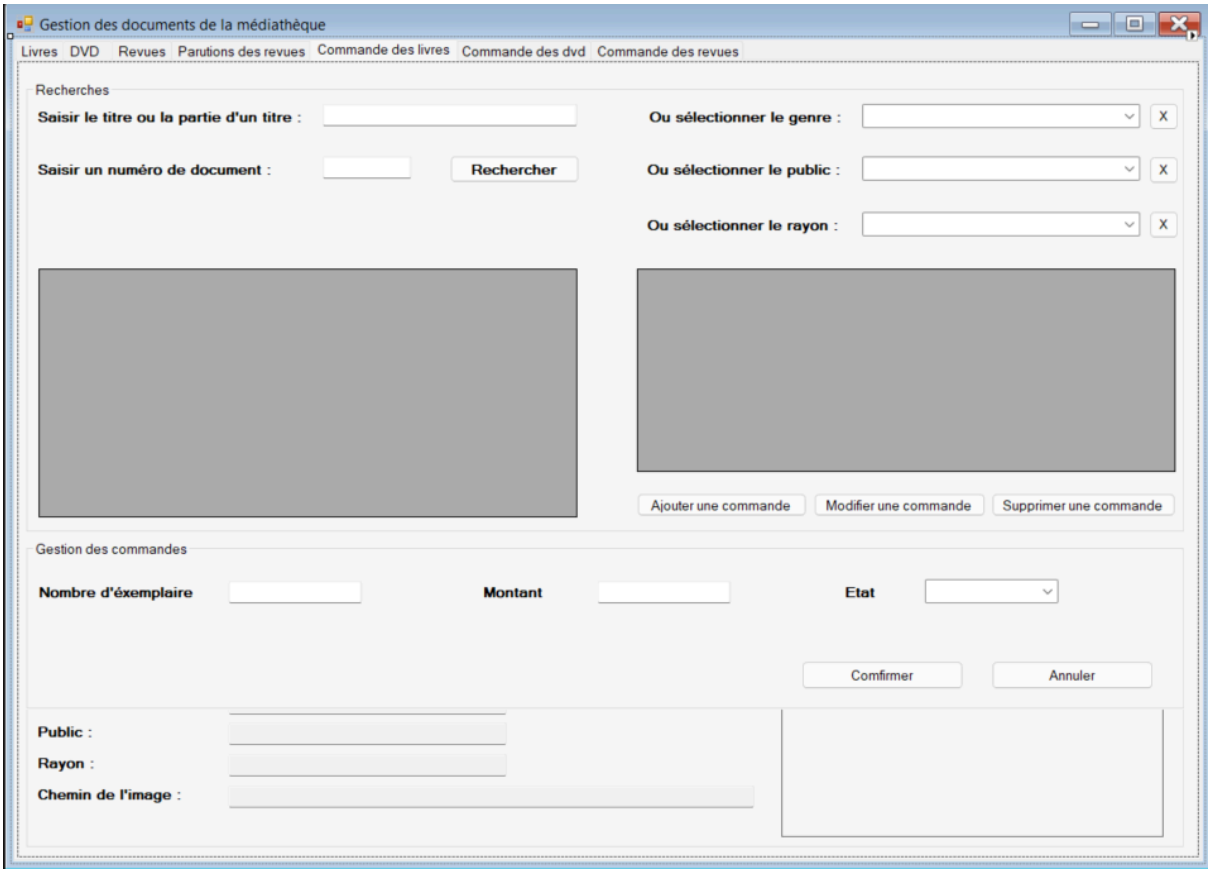


Partie de la BDD concernant la table suivi ajouté :



Concernant les explications suivantes, elles ne seront faites uniquement que pour la gestion des commandes des livres (les dvds ont les mêmes explications ainsi que la même maquette).

Maquette de l'onglet des commandes des livres :



Ajout d'un dataGridView à droite de la fenêtre qui contient les informations relatives aux commandes d'un livre sélectionné dans le dataGridView de gauche (contenant la liste de tous les livres). Sous le dgv de droite, ajout de 3 boutons : un bouton pour ajouter une

commande pour un livre sélectionné, un pour modifier une commande sélectionnée et un pour supprimer une commande sélectionnée.
Ajout d'un groupBox qui contient les informations des commandes à remplir, ce groupBox est caché et inaccessible au démarrage et n'est visible que lors d'un ajout ou une modification de commande.
Pour cette tâche il à été nécessaire d'utiliser les classes Livre.cs (pour les livres) et Dvd.cs (pour les dvd) principalement pour l'affichage des informations, une nouvelle classe Suivi.cs à été ajoutée afin de pouvoir gérer la gestion de suivi des commandes.

Pour les signatures des méthodes suivantes seules les fonctions créées seront abordées :

- RemplirListeCommande(List<Suivi>); cette méthode permet de remplir le dgv contenant les suivis par rapport à un livre sélectionné dans le dgv de gauche.
- dgvCommandeLivresListe_SelectionChanged(object, EventArgs); cette méthode permet lors de la sélection d'un livre (dgv de gauche) d'appeler les fonctions permettant d'afficher les suivis des commandes de ce livre. Permet aussi de verrouiller les boutons de modification et de suppression de suivi si ce livre n'en possède pas.
- btnAjoutCommande_Click(object, EventArgs); cette méthode permet lors de l'appui du bouton pour ajouter une commande, de verrouiller, de rendre invisible ou visible différents éléments graphiques.
- btnModifCommande_Click(object, EventArgs); cette méthode permet lors de l'appui du bouton de modification de faire comme le précédent bouton à la différence qu'il permet aussi de préremplir les champs à compléter.
- btnSupprCommande_Click(object, EventArgs); cette méthode à pour but de supprimer de la bdd les informations relatives à cette commande à la condition qu'elle ne soit pas livrée.
- btnAnnuler_Click(object, EventArgs); cette méthode permet lors du bouton d'annulation d'appeler la méthode endAction().
- btnComfirmer_Click(object, EventArgs); cette méthode permet lors de l'appui du bouton de confirmation d'ajouter ou de modifier dans la bdd un suivi de commande.
- endAction() permet de remettre les éléments graphiques dans leurs états d'origine.
- startAction() activé lors d'un ajout ou d'une modification, permet de rendre inaccessible le groupBox contenant les filtres et les listes, le groupBox permettant de visualiser les information est aussi caché et le groupBox avec les informations à remplir est lui, rendu visible et accessible.

Tâche 2 - Gérer les commandes de revues

Temps estimé 4h, temps réel 8h
Demande de la tâche : permettre de gérer les commandes des revues en ajoutant la possibilité de consulter les commandes, d'en ajouter, d'en supprimer et de les modifier. De plus ajouter une fenêtre au démarrage de l'application afin d'afficher les revues dont l'abonnement va bientôt se finir.

Capture d'écran du kanban :

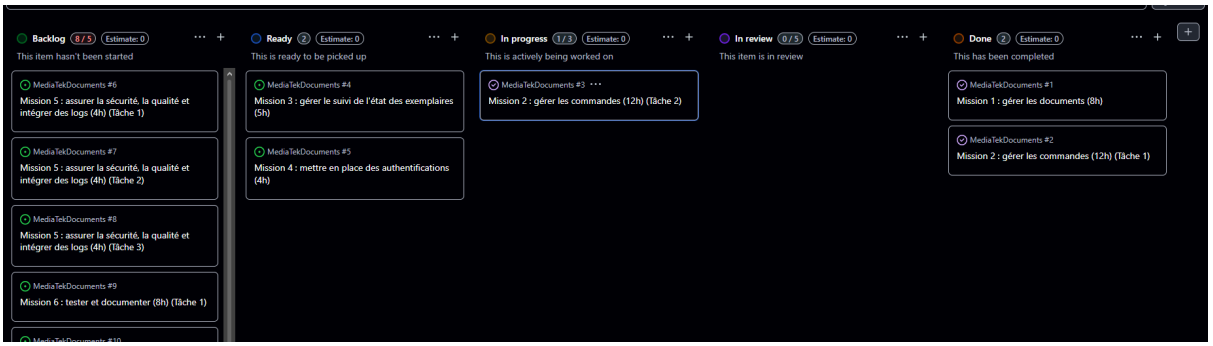
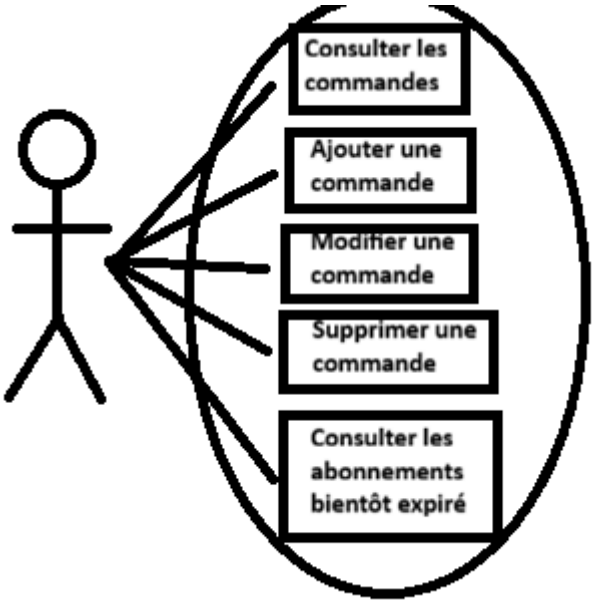
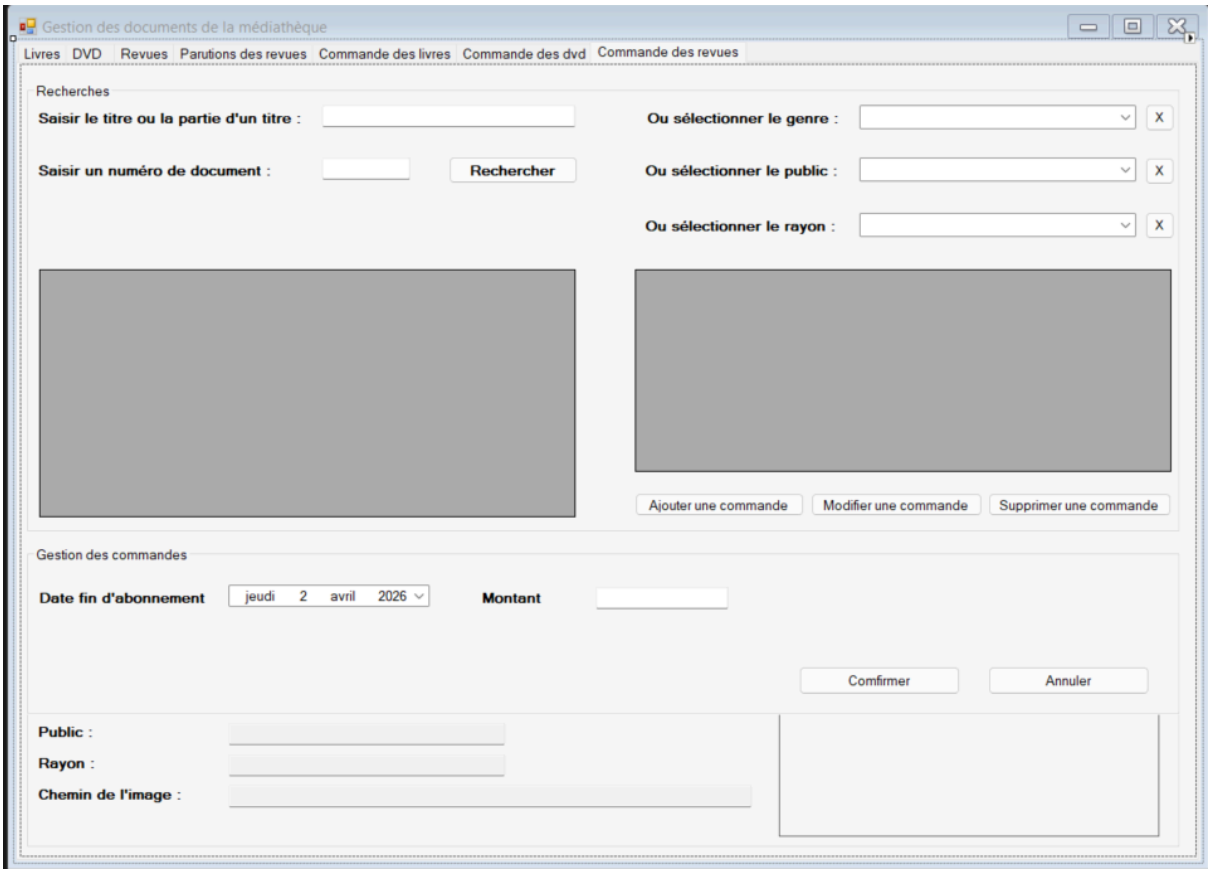


Diagramme d'utilisation de la tâche :

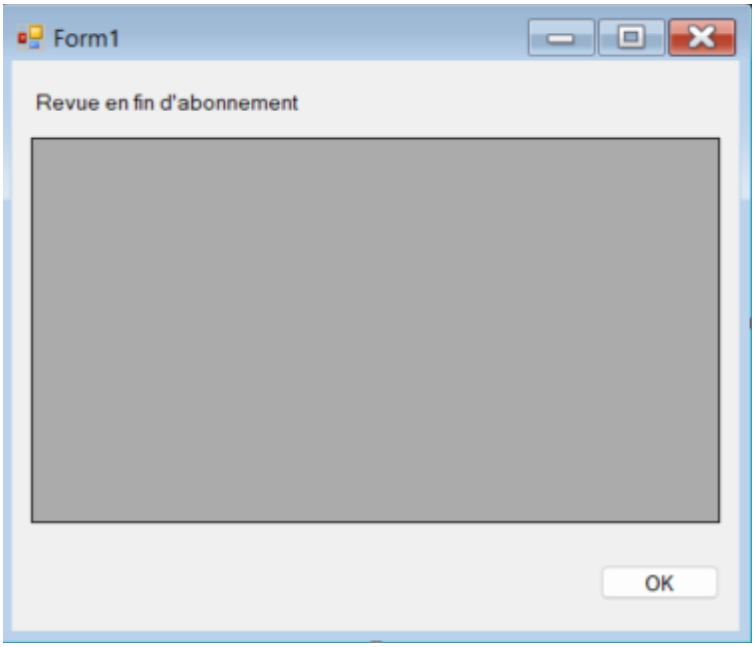


Maquette de l'onglet de gestion des commandes de revue :



Contient les mêmes éléments que pour la gestion de livre et dvd à l'exeption des éléments à ajouter ici la date de fin d'abonnement et le montant.

Maquette de la page de rappel des abonnements proche d'expirer :



Le dgv contient les informations relatives au documents proche d'expirer et le bouton permet de fermer cette fenêtre

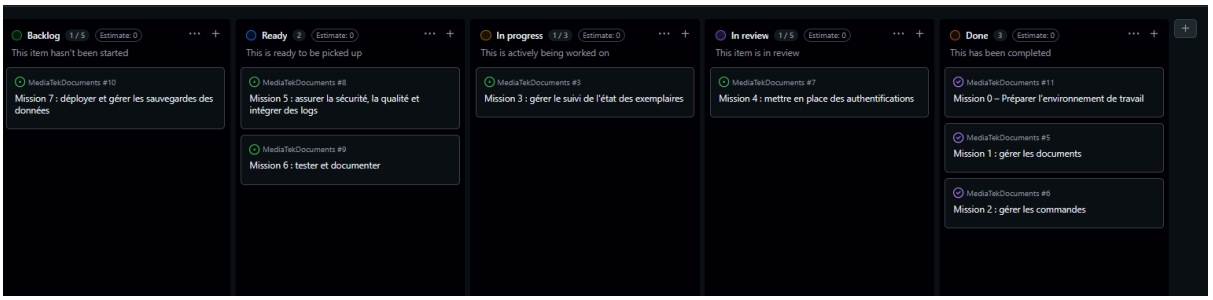
Pour cette tâche les classes impactées sont Revue.cs afin de gérer l'affichage des revues ainsi que la nouvelle classe Abonnement.cs qui contient les informations relatives à un abonnement et la nouvelle classe InfosExpiration.cs qui permet l'affichage des informations relatives aux abonnements bientôt expirées.

Les méthodes créées pour cette mission sont les mêmes que pour les livres et les dvd à la différence de leurs noms pour l'onglet de gestion des commandes de revues.
Les nouvelles méthodes pour la fenêtre des revue proche de la date d'expiration sont :

- Form1_Load(object, EventArgs) qui se déclenche lors de l'ouverture de la fenêtre et appelle la fonction RemplirRevueListeCommandeTab
- RemplirRevuesListeCommandeTab(List<InfosExpiration>) qui permet de remplir le dgv contenant les informations des revues qui vont expirer dans moins de 30 jours.
- btnOk_Click(object, EventArgs) qui permet de fermer la fenêtre et ainsi d'ouvrir la fenetre contenant la gestion des documents.

Mission 3 : gérer le suivi de l'état des exemplaires

Temps estimé : 5h. Temps réel : 15h.
Mise à jour du tableau kanban



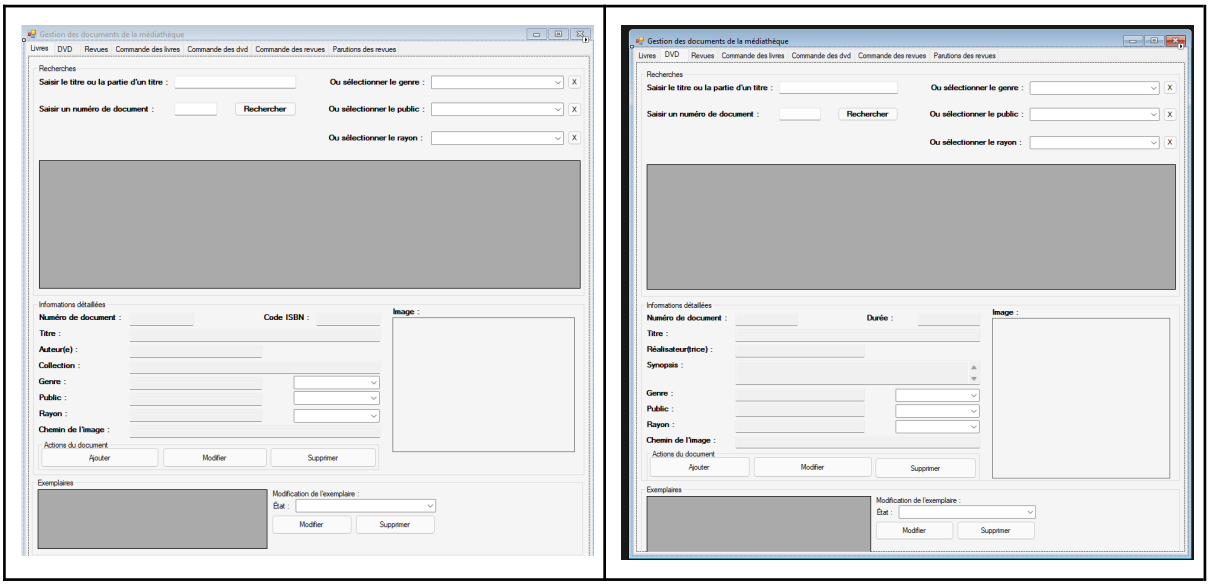
(Note : Les activités ayant été réalisées en quinconce, la mission 4 était avancée avant la mission 3.)

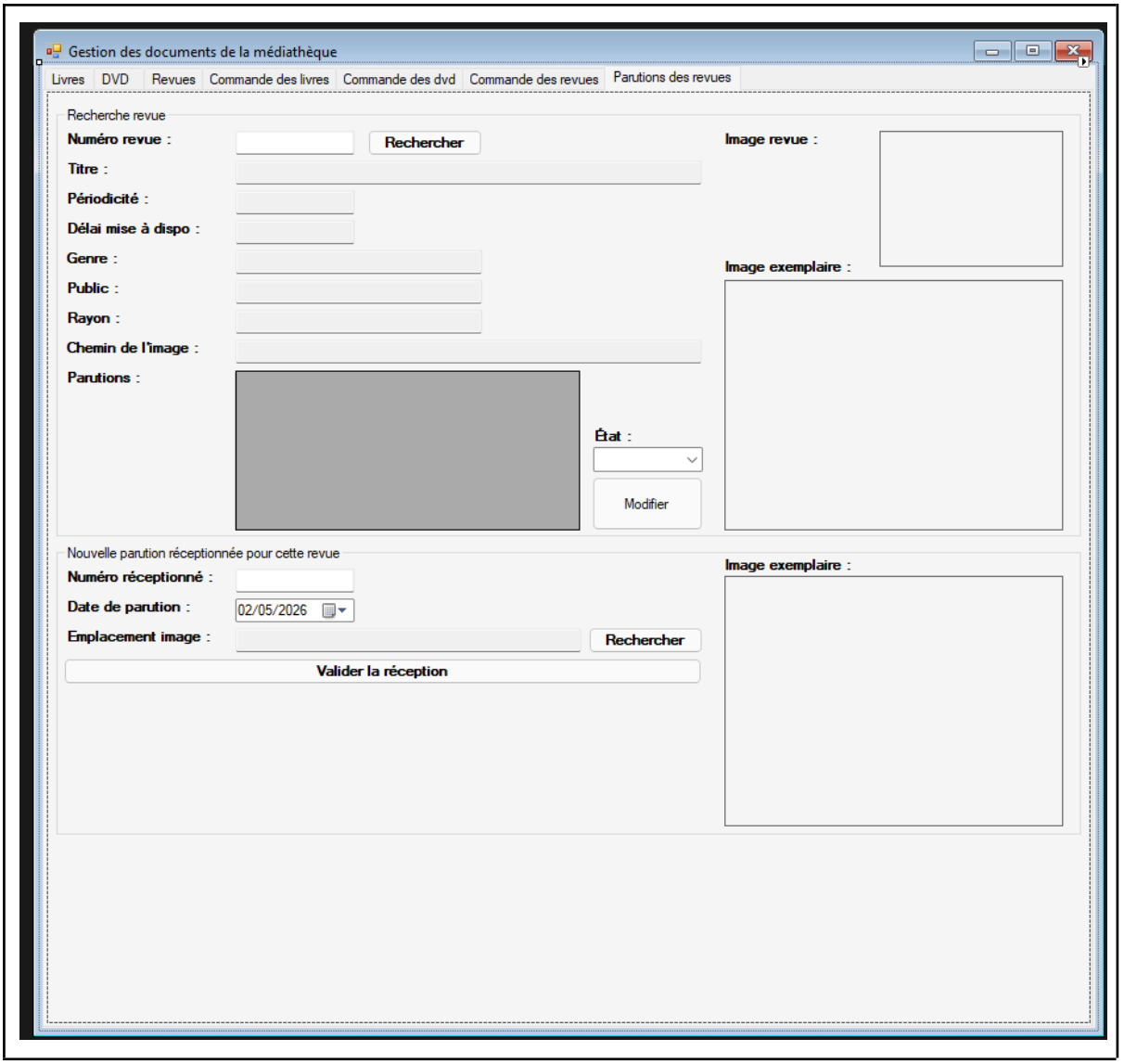
L'objectif de cette mission est de réaliser un système de gestion des exemplaires pour les pages Livres, DVD et Parution de revues.
Le tout est à ajouter en bas de l'application actuelle, après avoir augmenté l'espace vertical pris par l'application.
Le format de l'application est optimisé pour être vu entièrement sur un écran 1920x1080.

Les fonctionnalités à implémenter sont les suivantes :

1) Implémenter un tableau DataGridView pour afficher les exemplaires présents pour chaque élément de l'inventaire.
2) Ajouter une Combobox remplie des différents choix d'états disponibles.
3) Ajouter un bouton "Modifier" servant d'abord à débloquer la combobox, puis à valider notre choix d'Etat.
4) Ajouter un bouton supprimer qui supprimera, après confirmation, l'exemplaire sélectionné.

Voilà donc la vue modifiée pour les 3 onglets :





Ici, je ne présenterais seulement le code réalisé pour l'onglet Livres, étant donné que le procédé est similaire pour le reste.

Ajout de plusieurs fonctions dans Access.cs. Leur fonctionnement est décrit dans le commentaire.

```
169     /// <summary>
170     /// Récupère la liste de tous les exemplaires d'un document précis (avec son ID).
171     /// </summary>
172     /// <param name="idExemplaire"></param>
173     /// <returns></returns>
        1 référence | Paulygone 3D, il y a 2 jours | 1 auteur, 1 modification
174     public List<Exemplaire> GetAllExemplairesType(string idExemplaire)
175     {
176         String jsonIdDocument = ConvertToJson("id", idExemplaire);
177         List<Exemplaire> lesExemplaires = TraitementRecup<Exemplaire>(GET, "exemplaireglobal/" + jsonIdDocument, null);
178         return lesExemplaires;
179     }
180     /// <summary>
181     /// Récupère la liste de tous les Etats (neuf, usagé, etc.)
182     /// </summary>
183     /// <returns></returns>
        1 référence | Paulygone 3D, il y a 2 jours | 1 auteur, 1 modification
184     public List<Etat> GetAllEtats()
185     {
186         List<Etat> lesEtats = TraitementRecup<Etat>(GET, "etat", null);
187         return lesEtats;
188     }
```

```
229        /// <summary>
230        /// Modifie un exemplaire global
231        /// </summary>
232        /// <param name="exemplaire"></param>
233        /// <returns></returns>
           1 référence | Paulygone 3D, il y a 12 minutes | 1 auteur, 2 modifications
234        public bool ModifierExemplaire(Exemplaire exemplaire)
235        {
236            string jsonExemplaire = JsonConvert.SerializeObject(exemplaire, new CustomDateTimeConverter());
237            try
238            {
239                List<Exemplaire> liste = TraitementRecup<Exemplaire>(PUT, "exemplaireglobal", "id=null&champs=" + jsonExemplaire);
240                return (liste != null);
241            }
242            catch (Exception ex)
243            {
244                Console.WriteLine(ex.Message);
245            }
246            return false;
247        }
248        /// <summary>
249        /// Supprime un exemplaire global
250        /// </summary>
251        /// <param name="exemplaire"></param>
252        /// <returns></returns>
           1 référence | Paulygone 3D, il y a 2 jours | 1 auteur, 1 modification
253        public bool SupprimerExemplaire(Exemplaire exemplaire)
254        {
255            string jsonExemplaire = JsonConvert.SerializeObject(exemplaire, new CustomDateTimeConverter());
256            try
257            {
258                List<Exemplaire> liste = TraitementRecup<Exemplaire>(DELETE, "exemplaireglobal", "champs=" + jsonExemplaire);
259                return (liste != null);
260            }
261            catch (Exception ex)
262            {
263                Console.WriteLine(ex.Message);
264            }
265            return false;
266        }
```

Le pont est également réalisé avec la vue, en ajoutant ces fonctions au controller FrmMediatekController.cs.

Ensuite, dans l'API Rest, plus précisément dans MyAccessBDD.php, les nouvelles fontions ont été ajoutées avec le case "exemplaireglobal", étant donné que le case "exemplaire" est déjà utilisé pour le cas spécifique de l'insertion de revue.

```
protected function traitementSelect(string $table, ?array $champs) : ?array{
    switch($table){
        case "livre" :
            return $this->selectAllLivres();
        case "dvd" :
            return $this->selectAllDvd();
        case "revue" :
            return $this->selectAllRevues();
        case "exemplaire" :
            return $this->selectExemplairesRevue($champs);
        case "exemplaireglobal" :
            return $this->selectExemplairesType($champs);
        case "genre" :
```

Les fonctions suivantes sont ensuite ajoutées pour ajouter, modifier ou supprimer les données de la base de données.

```php
/**
 * Récupère tous les exemplaires d'un élément typé précis
 * @param array|null $champs
 * @return array|null
 */
private function selectExemplairesType(?array $champs) : ?array{
    if (empty($champs) || !array_key_exists('id', $champs)) {
        return null;
    }

    $idDocument = $champs['id'];
    $requete = "
        SELECT
            e.id,
            e.numero,
            e.dateAchat,
            e.photo,
            e.idEtat,
            et.libelle AS libelleEtat
        FROM
            exemplaire e
        JOIN
            etat et ON e.idEtat = et.id
        JOIN
            document d ON e.id = d.id
        WHERE
            e.id = :idDocument
        ORDER BY
            e.numero;
    ";
    $params = ['idDocument' => $idDocument];
    return $this->conn->queryBDD($requete, $params);
}
```

```php
private function updateExemplaireType($champs) : ?int{
    error_log("updateExemplaireType appelée avec : " . print_r($champs, true));
    if (empty($champs)) {
        return null;
    }


    $requete = "
        UPDATE exemplaire
        SET
            idEtat = :idEtat
        WHERE
            id = :id AND numero = :numero;
    ";

    $params = [
        'id' => $champs['Id'],
        'numero' => $champs['Numero'],
        'idEtat' => $champs['IdEtat']
    ];

    return $this->conn->updateBDD($requete, $params);
}
```

```php
private function deleteExemplaireType(array $champs) : ?int{
$requete = "
    DELETE FROM exemplaire
    WHERE
        id = :id AND numero = :numero;
";
$params = [
    'id' => $champs['Id'],
    'numero' => $champs['Numero']
];
    return $this->conn->updateBDD($requete, $params);
}
```

Ces fonctions sont utilisables peu importe le type d'exemplaire géré.

Enfin, les fonctionnalités sont ajoutées dans la vue. (Page suivante)

Fonction remplissant la liste des exemplaires :

```
/// <summary>
/// Remplis la liste des exemplaires de livres disponibles dans la base de données.
/// </summary>
/// <param name="exemplaire"></param>
2 références | Paulygone 3D, Il y a 18 heures | 1 auteur, 2 modifications
private void RemplirExemplairesLivresListe(List<Exemplaire> exemplaires)
{
    List<Etat> lesEtats = controller.GetAllEtats();

    bdgExemplairesLivresListe.DataSource = exemplaires;
    dgvExemplairesLivresListe.DataSource = bdgExemplairesLivresListe;

    dgvExemplairesLivresListe.Columns["id"].Visible = false;
    dgvExemplairesLivresListe.Columns["photo"].Visible = false;
    dgvExemplairesLivresListe.Columns["IdEtat"].Visible = false;

    if (dgvExemplairesLivresListe.Columns["LibelleEtat"] != null)
    {
        dgvExemplairesLivresListe.Columns.Remove("LibelleEtat");
    }

    DataGridViewTextBoxColumn etatColumn = new DataGridViewTextBoxColumn();
    etatColumn.HeaderText = "État";
    etatColumn.Name = "LibelleEtat";
    etatColumn.DataPropertyName = "IdEtat";
    dgvExemplairesLivresListe.Columns.Add(etatColumn);

    foreach (DataGridViewRow row in dgvExemplairesLivresListe.Rows)
    {
        if (row.DataBoundItem is Exemplaire exemplaire)
        {
            Etat etat = lesEtats.FirstOrDefault(e => e.Id == exemplaire.IdEtat);
            if (etat != null)
            {
                row.Cells["LibelleEtat"].Value = etat.Libelle;
            }
            else
            {
                row.Cells["LibelleEtat"].Value = "Inconnu";
            }
        }
    }

    dgvExemplairesLivresListe.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.AllCells;
}
```

Fonction rechargeant le DataGridView :

```
1 référence | Paulygone 3D, Il y a 51 minutes | 1 auteur, 3 modifications
private void dgvExemplairesLivresListe_ColumnHeaderMouseClick(object sender, DataGridViewCellMouseEventArgs e)
{
    string titreColonne = dgvExemplairesLivresListe.Columns[e.ColumnIndex].HeaderText;
    List<Exemplaire> sortedList = new List<Exemplaire>();
    switch (titreColonne)
    {
        case "Numero":
            sortedList = lesExemplairesLivres.OrderBy(o => o.Numero).ToList();
            break;
        case "DateAchat":
            sortedList = lesExemplairesLivres.OrderBy(o => o.DateAchat).ToList();
            break;
        case "État":
            sortedList = lesExemplairesLivres.OrderBy(o => o.IdEtat).ToList();
            break;
    }
    RemplirExemplairesLivresListe(sortedList);
    RemplirExemplairesLivresListeComplete();
}
```

Fonction de modification d'un exemplaire Livre du DataGridView :

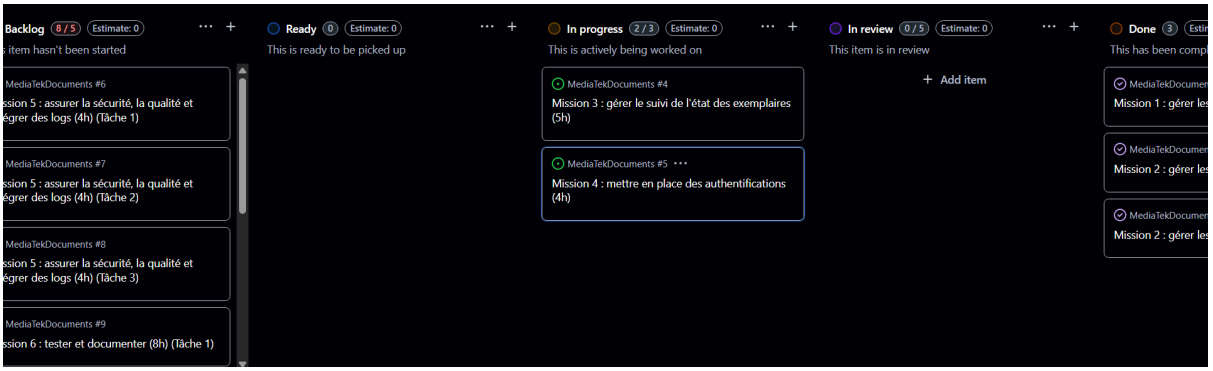
```
private void btnExemplaireLivreModifier_Click(object sender, EventArgs e)
{
    /// l'exemplaire sélectionné
    Exemplaire exemplaireSelected = (Exemplaire)bdgExemplairesLivresListe.List[bdgExemplairesLivresListe.Position];

    if (btnExemplaireLivreSupprimer.Enabled == true)
    {
        /// Phase 1
        cmbExemplaireLivreEtat.Enabled = true;
        btnExemplaireLivreSupprimer.Enabled = false;

        List<Etat> lesEtats = controller.GetAllEtats();

        cmbExemplaireLivreEtat.DataSource = lesEtats;
        cmbExemplaireLivreEtat.DisplayMember = "Libelle";
        cmbExemplaireLivreEtat.ValueMember = "Id";

        cmbExemplaireLivreEtat.SelectedIndex = 0;
    }
    else
    {
        if (cmbExemplaireLivreEtat.SelectedIndex >= 0)
        {
            Etat selectedEtat = (Etat)cmbExemplaireLivreEtat.SelectedItem;

            Exemplaire nvExemplaire = new Exemplaire(
                exemplaireSelected.Numero,
                exemplaireSelected.DateAchat,
                exemplaireSelected.Photo,
                selectedEtat.Id,
                exemplaireSelected.Id
            );

            if (controller.ModifierExemplaire(nvExemplaire) == true)
            {
                MessageBox.Show("Exemplaire modifié avec succès.");
                cmbExemplaireLivreEtat.Enabled = false;
                btnExemplaireLivreSupprimer.Enabled = true;

                lesExemplairesLivres = controller.GetAllExemplairesType(exemplaireSelected.Id);
                RemplirExemplairesLivresListeComplete();
            }
            else
            {
                MessageBox.Show("Erreur lors de la modification.", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
        else
        {
            MessageBox.Show("Veuillez sélectionner un état.", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}
```

Et enfin, la fonction de suppression d'un exemplaire Livre du DataGridView :

```
1 référence | Paulygone 3D, Il y a 54 minutes | 1 auteur, 3 modifications
private void btnExemplaireLivreSupprimer_Click(object sender, EventArgs e)
{
    if (dgvExemplairesLivresListe.SelectedRows.Count == 1)
    {
        Exemplaire exemplaireSelected = (Exemplaire)bdgExemplairesLivresListe.List[bdgExemplairesLivresListe.Position];

        var request = MessageBox.Show("Souhaitez-vous supprimer l'exemplaire n°" + exemplaireSelected.Numero.ToString() + " ?",
                        "Confirmation", MessageBoxButtons.YesNo, MessageBoxIcon.Warning);
        if (request == DialogResult.Yes)
        {
            btnExemplaireLivreModifier.Enabled = false;
            btnExemplaireLivreSupprimer.Enabled = false;

            Exemplaire nvExemplaire = new Exemplaire(
                exemplaireSelected.Numero,
                exemplaireSelected.DateAchat,
                exemplaireSelected.Photo,
                exemplaireSelected.IdEtat,
                exemplaireSelected.Id
            );

            if (controller.SupprimerExemplaire(nvExemplaire) == true)
            {
                btnExemplaireLivreModifier.Enabled = true;
                btnExemplaireLivreSupprimer.Enabled = true;
                MessageBox.Show("Exemplaire supprimé avec succès.");
                lesExemplairesLivres = controller.GetAllExemplairesType(exemplaireSelected.Id);
                RemplirExemplairesLivresListeComplete();
            }
            else
            {
                MessageBox.Show("Erreur lors de la suppression.", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
        }
        else
        {
            VisibiliteChamps(true, "Supprimer");
        }
    }
    else
    {
        MessageBox.Show("Veuillez ne sélectionner qu'une entrée.", "Erreur", MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
}
```

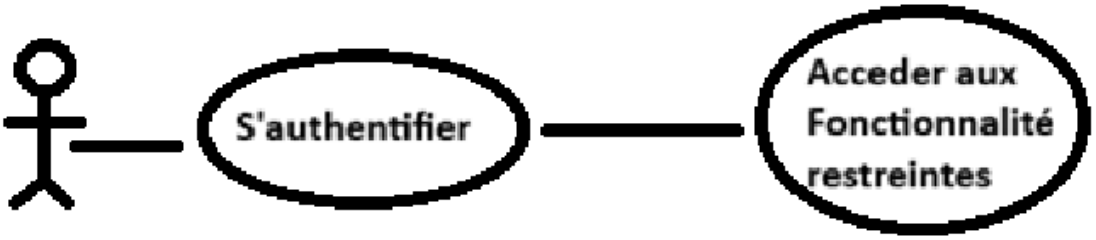
Mission 4 : Mettre en place des authentifications

Temps estimé : 4h, temps réel 4h
Mise à jour du tableau kanban

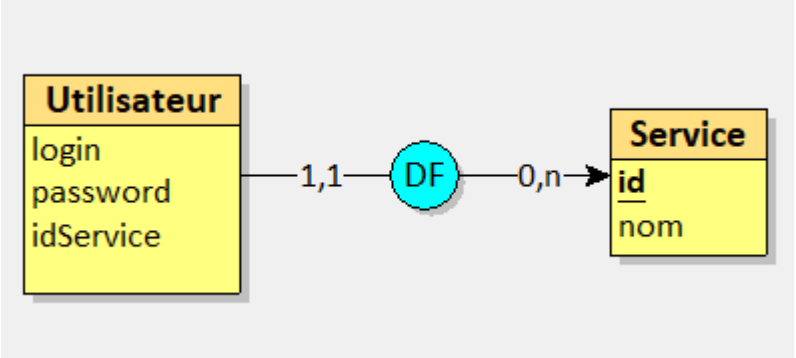


Demande de la tâche : Ajouter un formulaire s'affichant au démarrage et permettant de s'authentifier, de plus après l'authentification restreindre l'accès à certain éléments de l'application en fonction du type de service des employés.

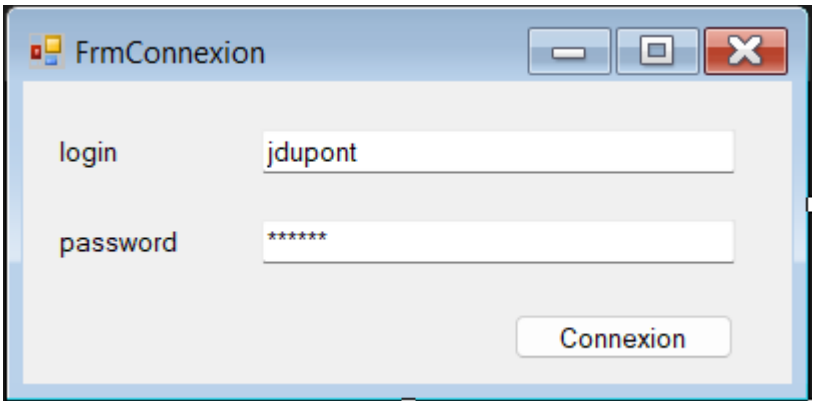
Diagramme d'utilisation de la tâche :



MCD de la tâche :



Maquette du formulaire :



Le formulaire est constitué de 2 textbox, un pour le nom est un pour le mot de passe, le bouton sert à envoyer les informations et à les comparer avec la bdd

Ajout d'une nouvelle classe Connexion.cs contenant, le login, password, et id renvoyé par la bdd.
Ajout d'une méthode Verrouillage() qui cache et bloque l'accès à certains éléments graphiques dans le cas ou l'id de la personne connecté est "prets" (l'administratif et l'administrateur ont tous les accès et celui de la culture aucun). les restrictions sont les suivantes :

- Inaccessibilité visuelle et interactive des éléments permettant de modifier, ajouter ou supprimer des éléments.
- Suppression des onglets de gestions

Mission 5 : Assurer la sécurité, la qualité et intégrer des logs

Tâche 1 - Corriger des problèmes de sécurité

Problème 1 :
Demande de la tâche :
Sécuriser l'accès à l'api en cachant le couple login:pwd écrit en dur.
Kanban de la tâche :

Temps estimé 1h : temps réel 1h

Pour résoudre ce problème, il à fallu créer dans creer dans app.config une section appSetting contenant une clé-valeur contenant les informations d'accès à l'API. Ce paramètre est appelé dans la classe Access.cs afin de pouvoir avoir accéder aux paramètres de connexion.

Tâche 2 - Contrôler la qualité

Demande de la tâche : nettoyer le code ajouté avec les règles d'écriture standard
Kanban de la tâche en cours :

Temps estimé 1h temps réel 1h30
Extrait des problèmes :

Tâche 3 - Intégrer des logs

Demande de la tâche : Déplacer l'url d'accès à l'API écrit en brut dans le fichier config et ajouter des logs dans la classe Access.cs à chaque affichage console.
Kanban de la tâche en cours :

Temps estimé : 1h, temps réel : 1h
Code permettant de configurer les logs :

```
private Access()
{
    Log.Logger = new LoggerConfiguration()
        .MinimumLevel.Verbose()
        .WriteTo.Console()
        .WriteTo.File("logs/log.txt", rollingInterval: RollingInterval.Day)
        .WriteTo.File("logs/errorlog.txt", restrictedToMinimumLevel: Serilog.Events.LogEventLevel.Information)
        .CreateLogger();
```

Exemple de code d'ajout de log :

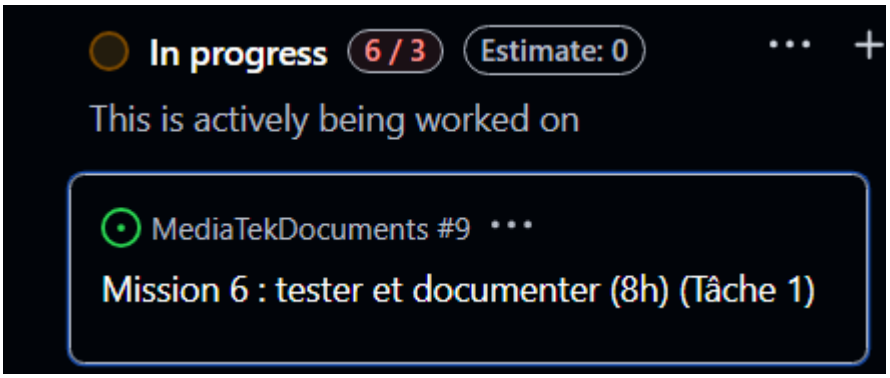
```
Log.Fatal(e, "Access.Access() - Erreur lors de la tentative de connexion à l'API : {0}", e.Message);
```

Mission 6 : Tester et documenter

Tâche 1 - Gérer les tests

Demande de la tâche : Écrire des test unitaire afin de tester les classes du package Model ainsi que des tests dans POSTMAN afin de tester les fonctionnalités d'accès à la BDD.

Temps estimé 5h, temps réel 5h
Kanban de la tâche :



Exemple de code et de résultat :

```
public void ConstructeurTest()
{
    DateTime dateCommande = new DateTime(2026, 04, 29);
    DateTime dateFin = new DateTime(2027, 04, 29);
    string idRevue = "10002";
    double montant = 120.0;
    string id = "10";

    Abonnement abonnement = new Abonnement(dateCommande, dateFin, idRevue, montant, id);

    Assert.AreEqual(dateCommande, abonnement.DateCommande);
```

AbonnementTest (1)	4 ms
ConstructeurTest	4 ms

GET http://localhost/rest_mediatekdocuments/livre

Docs | Params | Authorization | Headers (8) | Body | Scripts | Settings

Pre-request | Post-response

```
pm.test("Status code is 200", function () {
    pm.response.to.have.status(200);
});
```

Packages

Body | Cookies | Headers (7) | Test Results (1/1) — 200 OK · 541 ms · 6.52 KB

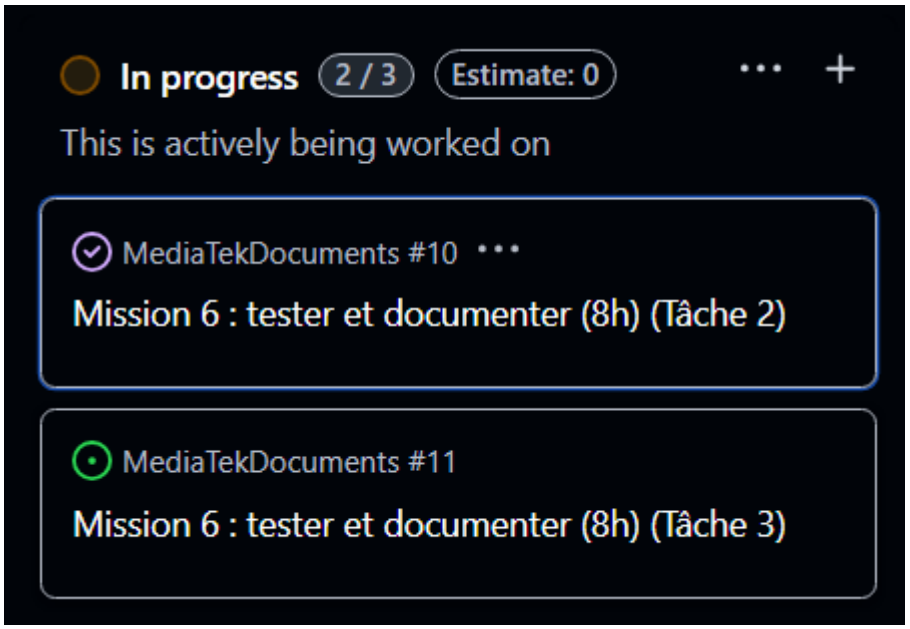
JSON | Preview | Visualize


```
{
    "code": 200,
    "message": "OK",
    "result": [
        {
            "id": "00017",
            "ISBN": "",
            "auteur": "Philippe Masson",
            "titre": "Catastrophes au Brésil",
            "image": "",
            "collection": "",
            "idrayon": "JN002",
            "idpublic": "00004",
            "idgenre": "10014",
            "genre": "Policier",
            "lePublic": "Ados",
            "rayon": "Jeunesse romans"
        },
        {
            "id": "00007",
            "ISBN": "6541236987541",
```


Tâche 2 - Générer la documentation technique

Demande de la tâche : Ajouter les commentaires normalisé pour l'API Rest et le projet C#, puis générer la documentation technique.

Kanban de la tâche :



Temps estimé 1h, temps réel 2h30

Exemple de la documentation générée pour l'API et le projet :

Classes

AccessBDD

Classe qui sollicite ConnexionBDD pour l'accès à la BDD MySQL Elle contient les méthodes appelées par Controle et les méthodes abstraites que MyAccessBDD doit redéfinir pour construire les requêtes

Connexion

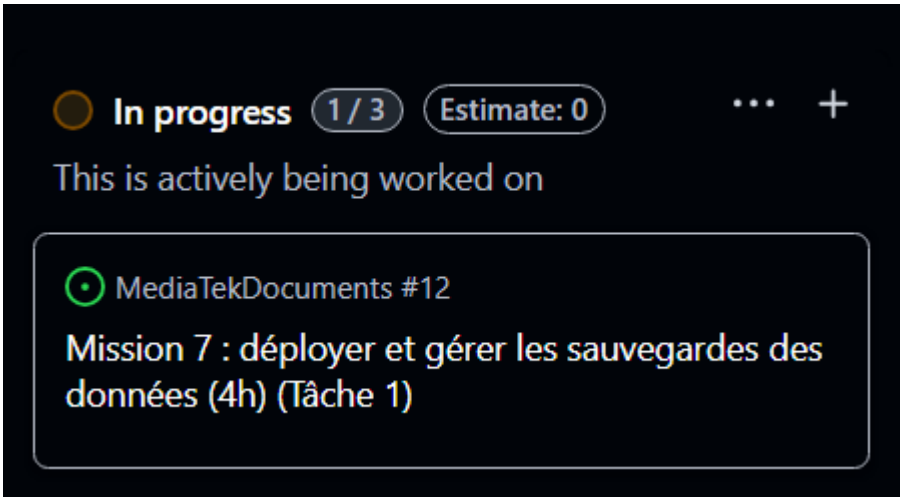
Classe de connexion à la BDD MySQL (singleton) et d'exécution des requêtes en

```
▼<members>
  ▼<member name="T:MediaTekDocuments.controller.FrmMediatekController">
      <summary> Contrôleur lié à FrmMediatek </summary>
    </member>
  ▼<member name="F:MediaTekDocuments.controller.FrmMediatekController.access">
      <summary> Objet d'accès aux données </summary>
    </member>
  ▼<member name="M:MediaTekDocuments.controller.FrmMediatekController.#ctor">
      <summary> Récupération de l'instance unique d'accès aux données </summary>
    </member>
```

Mission 7 : déployer et gérer les sauvegardes des données

Tâche 1 - Déployer le projet

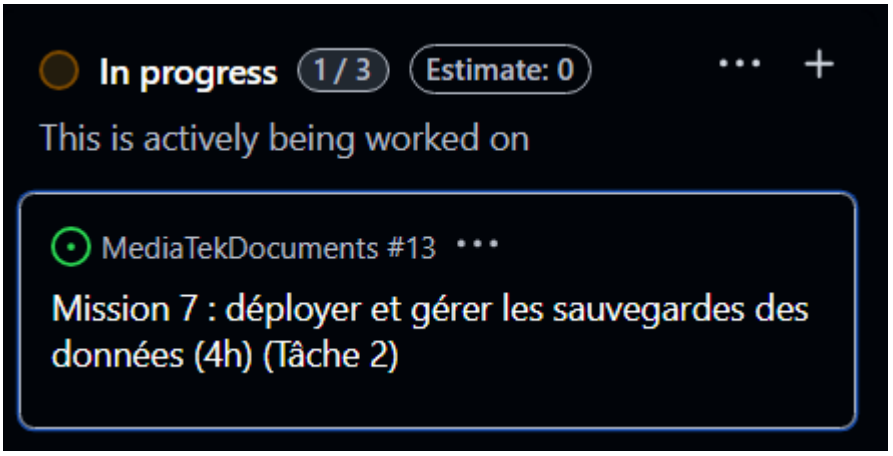
Temps estimé 3h : temps réel 4h
Demande de la tâche : publier l'API et la BDD et créer un installeur du projet
Kanban de la tâche :



Explication des étapes :
On commence par transférer chez l'hébergeur l'API ainsi que la base de données. Ensuite on va modifier dans le fichier .env les informations de connexion à la BDD et de connexion à l'API grâce aux informations fournies par l'hébergeur. Dans le projet C# on va aussi modifier le fichier App.config avec les informations d'accès à l'API, à savoir le couple login:pwd et l'adresse de l'API.

Tâche 2 - Gérer les sauvegardes des données

Temps estimé : 1h temps réel : 30min
Kanban de la tâche :



Demande de la tâche : Programmer une sauvegarde journalière automatisé de la bdd
Pour ce faire il faut créer le fichier de script nécessaire grace au code suivant :

```
#!/bin/sh DATE=`date -I` find /home/[logincompte]/savebdd/bdd* -mtime -1
```

```
-exec rm {} \; mysqldump -u [loginuserbdd] -p[pwduserbdd] --databases
[namebdd] --single-transaction | gzip >
/home/[logincompte]/savebdd/bddbackup_${DATE}.sql.gz
```

Il faut ensuite mettre ce fichier au format linux et le transférer à l'hébergeur enfin il faut faire une tâche Cron pour automatiser l'exécution de ce script. Malheureusement pour ce projet l'hébergeur ne permet pas de pouvoir faire des backups de manière gratuite.

Bilan

L'application Mediatek_Documents a donc pu être maintenue et améliorée correctement, dans le respect de la méthode MVC, en mobilisant tous les apprentissages de BTS SIO.
Chaque demande du cahier des charges est remplie, du contrôle de chaque document, aux commandes, en passant par les exemplaires, l'authentification, et ainsi de suite.
Les tests unitaires ont été réalisés et la documentation technique est générée et disponible à tous.
L'application est déployée correctement et fonctionne à distance, ce qui permettra à tous les employés de MediaTek86 de réaliser leur travail dans les meilleures conditions possibles grâce à ce logiciel développé sur mesure.

Comment installer l’application en local ?

Pour installer l’application localement il faut dans un premier temps télécharger le code de l’application C#. Ensuite vous devrez avoir WampServer d’installé puis vous devrez ajouter dans le dossier www l’API trouvable sur le github de l’application, puis exécuter la commande “composer install” dans une fenêtre en mode admin. Enfin dans phpmyadmin télécharger et importer le fichier sql contenant la base de donnée. Il y a dans cette base un table ‘utilisateur’ le couple login:password vous permettra d'accéder à l’application et à ses fonctionnalités.